S2. Azure pipelines

M13. DevOps

ServiceNow
Faculdade de Ciências da Universidade de Lisboa
UPskill – Programa Digital SKILLS & JOBS (2025/26)
Mário Calha

Conteúdo

- Azure pipelines
- Azure DevOps

Pipeline

1. Code is pushed to a source control repository, such as Git.
2. Azure Pipelines is triggered by this code change and performs a series of steps as defined in the pipeline.
3. **Build**: The code is compiled and turned into artifacts, such as executables or Docker images.
4. **Test**: Automated tests are run on the artifacts to verify that the application works as expected.
5. **Deploy**: If the tests pass, the artifacts are deployed to the desired environment, such as a production server or a staging environment, for further testing

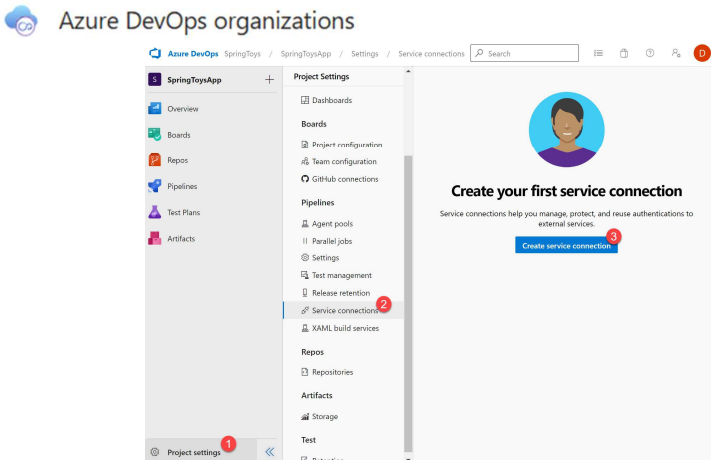
https://learn.microsoft.com/en-us/azure/devops/pipelines

Criação de pipelines

- **Classic Editor**
  - A graphical interface that allows users to create pipelines using a visual designer. The Classic Editor is well suited to users who prefer a visual approach and do not have extensive knowledge of scripting or coding.
- **YAML Pipelines**
  - Azure DevOps supports defining pipelines using YAML files, which describe the entire pipeline in a declarative manner. YAML pipelines offer greater flexibility and version control capabilities compared to the Classic Editor. With YAML, you define the pipeline's stages, jobs, steps, and dependencies using plain text files that can be stored in source control. This approach is popular among developers who prefer to have their pipeline configurations as code and want to take advantage of features such as code reviews, branching, and versioning.

Configuração Azure DevOps

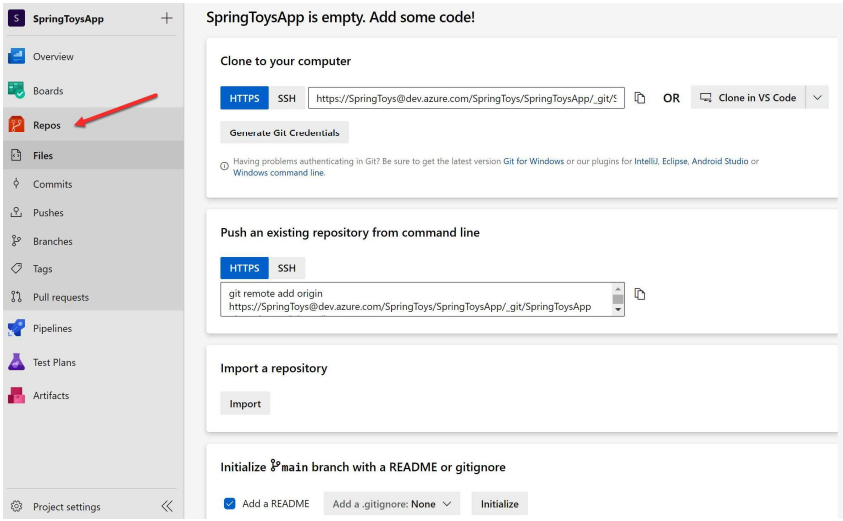
- Criar organização DevOps
- Criar Projeto
- Configurar projeto
  - Service connections
    - Create service connection
      - Azure Resource Manager
        - Subscription



https://learn.microsoft.com/en-us/azure/devops/pipelines/library/service-endpoints?view=azure-devops&tabs=yaml#common-service-connection-types?WT.mc_id=AZ-MVP-5000671

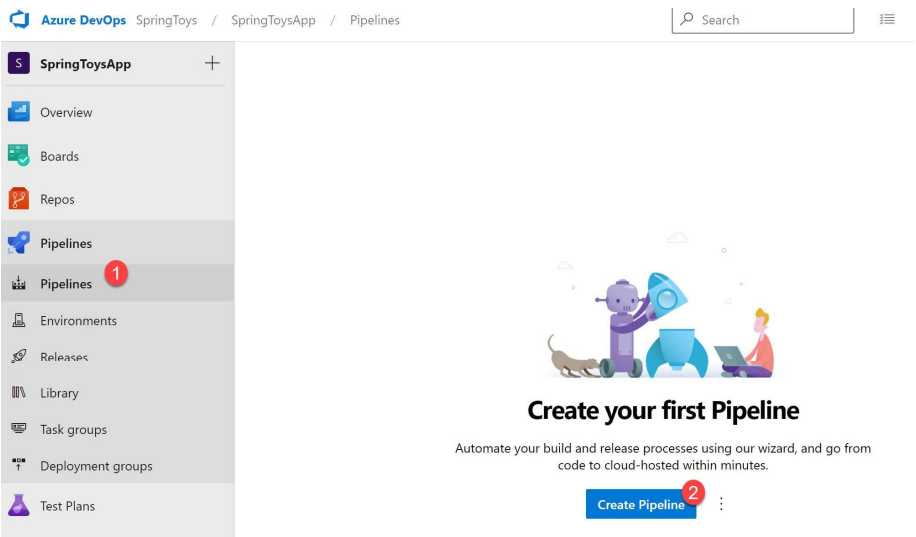
Configuração Azure Repos

- Suporte para Git
  - inclui pull requests, code reviews ...



Configuração de pipeline Azure

- Pipelines
  - Create pipeline



YAML de base

- Conteúdo inicial

New pipeline

Review your pipeline YAML

SpringToysApp / **azure-pipelines.yml** *

```yaml
# Starter pipeline
# Start with a minimal pipeline that you can customize to build and deploy your code.
# Add steps that build, run tests, deploy, and more:
# https://aka.ms/yaml

trigger:
- master

pool:
  vmImage: ubuntu-latest

steps:
- script: echo Hello, world!
  displayName: 'Run a one-line script'

- script: |
    echo Add other tasks to build, test, and deploy your project.
    echo See https://aka.ms/yaml
  displayName: 'Run a multi-line script'
```

YAML final

- Especificar as tarefas e configurações para cada passo de um trabalho (job)
- Azure fornece tarefas
  - build
  - test runners
  - deployment
- Configuração de
  - condições
  - variáveis
  - triggers

Review your pipeline YAML

SpringToysApp / SpringToysApp-CI-AutomationAccount.yml *

```yaml
trigger:
  branches:
    include:
    - refs/heads/master
jobs:
- job: Job_1
  displayName: Agent job 1
  pool:
    vmImage: windows-2019
  steps:
  - checkout: self
    fetchDepth: 1
    Settings
  - task: PublishBuildArtifacts@1
    displayName: Tier1-CI-AutomationAccount
    inputs:
      PathtoPublish: Tier1/AutomationAccount
      ArtifactName: AutomationAccount
      ......
...
```


Azure pipeline

- Build and publish a Python app
  https://learn.microsoft.com/en-us/azure/devops/pipelines/ecosystems/python?view=azure-devops
- Pipeline

CI trigger: main branch

build platform

versions that will be tested

pre-packaged script

```yaml
trigger:
- main
pool:
  vmImage: ubuntu-latest
strategy:
  matrix:
    Python310:
      python.version: '3.10'
      (...)
steps:
  - task: UsePythonVersion@0
    inputs:
      versionSpec: '$(python.version)'
    displayName: 'Use Python $(python.version)'

  - script: |
      python -m pip install --upgrade pip
      pip install -r requirements.txt
    displayName: 'Install dependencies'
```

```yaml
(...)
- task: ArchiveFiles@2
    displayName: 'Archive files'
    inputs:
      rootFolderOrFile: $(System.DefaultWorkingDirectory)
      includeRootFolder: false
      archiveType: zip
      archiveFile: $(Build.ArtifactStagingDirectory)/$(Build.BuildId)-$(python.version).zip
      replaceExistingArchive: true

  - task: PublishBuildArtifacts@1
    inputs:
      PathtoPublish: '$(Build.ArtifactStagingDirectory)'
      ArtifactName: 'drop'
      publishLocation: 'Container'

  - script: |
      pip install pytest pytest-azurepipelines
      pytest
    displayName: 'pytest'
```

Pipeline tasks

- Azure Pipelines task reference
  https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference
- Grupos de tarefas
  - Build
  - Deploy
  - Package
  - Test
  - Tool
  - Utility
  - Open source (on GitHub)

Pipeline conditions

- By default, a job or stage runs if it doesn't depend on any other job or stage, or if all its dependencies completed and succeeded.
- By default, a step runs if nothing in its job failed yet and the step immediately preceding it completed.
- You can override or customize this behavior by forcing a stage, job, or step to run even if a previous dependency fails, or by specifying a custom condition.
  - `succeededOrFailed()`
  - `always()`
  - `failed()`

  https://learn.microsoft.com/en-us/azure/devops/pipelines/process/conditions

Pipeline variables

- The most common use of variables is to define a value that you can then use in your pipeline. All variables are strings and are mutable.

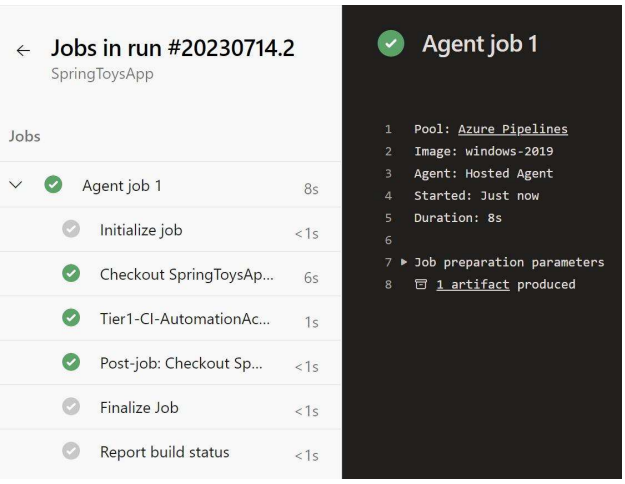
Syntax	Example	When is it processed?	Where does it expand in a pipeline definition?	How does it render when not found?
macro	`$(var)`	runtime before a task executes	value (right side)	prints `$(var)`
template expression	`${{ variables.var }}`	compile time	key or value (left or right side)	empty string
runtime expression	`$[variables.var]`	runtime	value (right side)	empty string

https://learn.microsoft.com/en-us/azure/devops/pipelines/process/variables

Pipeline triggers

- Continuous Integration triggers
- Pull Request validation triggers
- Comment triggers, Scheduled triggers, Pipeline triggers
- …

https://learn.microsoft.com/en-us/azure/devops/pipelines/build/triggers?view=azure-devops

Estado do pipeline

- O YAML é validado quando é guardado
- Sempre que é executado o estado é registado e é possível saber o resultado de cada passo